04. Preprocessing & Feature Engineering

Build a combined textual representation from structured metadata.

Project: Movie Recommender System

Source archive supplied in this conversation: Movie-recommender-system-main.zip | Notebook: notebook.ipynb | App: app.py

Parsing JSON-like columns

The genres, keywords and cast columns contain stringified lists of dictionaries. ast.literal_eval safely parses Python literal structures without executing arbitrary code like eval. The convert() helper extracts each dictionary's name field into a list. The crew parser selects only entries where job == 'Director'.

Feature construction

- genres → list of genre names.
- keywords → list of keyword names.
- cast → list of cast names, then only first three retained.
- crew → director names only.
- overview → split into whitespace-separated tokens.
- collapse() removes spaces inside each name, e.g. a multi-word name becomes a single token.

Tag aggregation

The notebook concatenates overview + genres + keywords + cast + crew into one list called tags, then joins the list into one string per movie. This is a simple feature-engineering strategy: all metadata sources share one vocabulary and contribute to a single document representation.

Vectorization

CountVectorizer(max_features=5000, stop_words='english') builds a vocabulary of up to 5,000 tokens and converts each movie tag string into a sparse term-count vector. The notebook calls .toarray(), producing a dense matrix with shape (4,806, 5,000). Count vectors represent term frequency, not semantic meaning; synonyms and related phrases are not inherently close unless they share tokens.

Preprocessing gaps and alternatives

- No explicit stemming/lemmatization, n-gram configuration, min_df or max_df tuning.
- No field-specific weighting: a frequent cast or overview token may have the same raw count contribution as a genre token.
- Use TF-IDF to downweight words appearing in many movies.
- Use field-aware weighting so genre/director/cast signals can be controlled independently.
- Consider sentence/document embeddings for semantic similarity; validate carefully for domain fit and explainability.
- Use stable IDs and preserve a clean mapping between vector row, movie_id and title.

Code-level caution

The notebook defines convert3() to keep the first three entries, but the actual cast pipeline calls convert() and then slices movies['cast'] to the first three. This achieves the intended top-three cast selection, but the unused helper can be removed for clarity.